



Adding a Database Layer

AWS Academy Cloud
Architecting



Introduction

Adding a Database Layer

Module objectives

This module prepares you to do the following:

- Compare database types and services offered by Amazon Web Services (AWS).
- Explain when to use Amazon Relational Database Service (Amazon RDS).
- Describe the advanced features of Amazon RDS.
- Explain when to use Amazon DynamoDB.
- Identify AWS purpose-built database services.
- Describe how to migrate data into AWS databases.
- Design and deploy a database server.
- Use the AWS Well-Architected Framework principles when designing a database layer.

Module overview

Presentation sections

- Database layer considerations
- Amazon RDS
- Amazon RDS Proxy connection management
- Amazon DynamoDB
- Purpose-built databases
- Migrating data into AWS databases
- Applying AWS Well-Architected Framework principles to the database layer

Demo

- Amazon RDS Automated Backup and Read Replicas

Knowledge checks

- 10-question knowledge check
- Sample exam question

Hands-on labs in this module

Guided lab

- Creating an Amazon RDS Database

Challenge (Café) lab

- Migrating a Database to Amazon RDS



As a cloud architect
designing a database
layer:



- I need to evaluate the available database options before selecting a data management solution so that I can optimize performance.
- I need to secure my infrastructure effectively so that data is durable and safe from threats.



Database layer considerations

Adding a Database Layer

Database considerations



Scalability

How much throughput is needed?
Will it scale?

Storage requirements

Will the database need to be sized in gigabytes, terabytes, or petabytes of data?

Data characteristics

What is your data model? What are your data access patterns?
Do you need low latency?

Durability

What level of data durability, availability, and recoverability is required?
Do regulatory obligations apply?

Relational and non-relational databases

Features	Relational Databases	Non-Relational Databases
Structure	Tabular form of columns and rows	Variety of structure models (key-value pairs, document, or graph-based)
Schema	Strict schema rules	Flexible schemas
Benefits	Ease of use, data integrity, reduced data storage, and common language (SQL)	Flexibility, scalability, and high performance
Use Case	When migrating an on-premises relational workload or if your workload involves online transactional processing	When a caching layer is needed to improve read performance, when storing JSON documents, or when a single digit millisecond data retrieval is needed
Optimization	Optimized for structured data stored in tables; supports complex one-time queries through joins	Optimized for fast access to structured, semi-structured, or unstructured data with high read and write throughput

Amazon database options

Relational databases



Amazon RDS

Managed database service that provides seven familiar database engines to choose from, including Amazon Aurora

Non-relational databases



Amazon
DynamoDB



Amazon
Neptune



Amazon
ElastiCache

Variety of services designed for databases such as key-value, graph, and in-memory

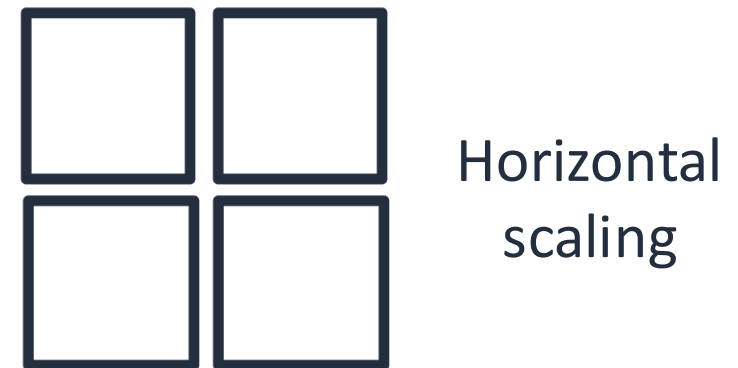
Less responsibility with managed AWS database services

Tasks	Host Database on Premises	Host Database in Amazon EC2	Host Database in a Managed AWS Database Service
Power, HVAC, and Network	X		
Rack and Stack	X		
Server Maintenance	X		
OS Installation	X		
OS Patches	X		
Database Installation	X	X	
Database Patches	X	X	
Database Backups	X	X	
High Availability	X	X	
Scaling	X	X	
Application Optimization	X	X	X

Database capacity planning

Designing with the end goal in mind

1. Analyze current storage capacity.
2. Predict capacity requirements.
3. Determine if horizontal scaling, vertical scaling, or a combination is needed.



Key takeaways: Database layer considerations



- When you choose a database, consider scalability, storage requirements, data characteristics, cost, and durability requirements. Database capacity planning is also a consideration.
- Relational databases have strict schema rules, scale vertically, provide data integrity, and work well for structured data stored in tables.
- Non-relational databases have flexible schemas, scale horizontally, provide higher scalability and flexibility, and work well for semi-structured and unstructured data.
- With AWS managed database services, you are responsible only for optimizing your application.



Amazon RDS

Adding a Database Layer

Amazon Relational Database Service



Amazon RDS

- Is a managed relational database service to deploy and scale relational databases
- Supports multiple database engines
- Uses Amazon Elastic Block Store (Amazon EBS) volumes for database and log storage

Benefits of Amazon RDS



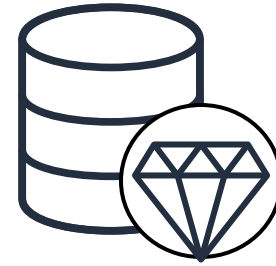
Lower administrative burden

You don't need to provision your infrastructure or install and maintain database software.



Highly scalable

You can scale up or down the compute and memory resources powering your deployment.



Available and durable

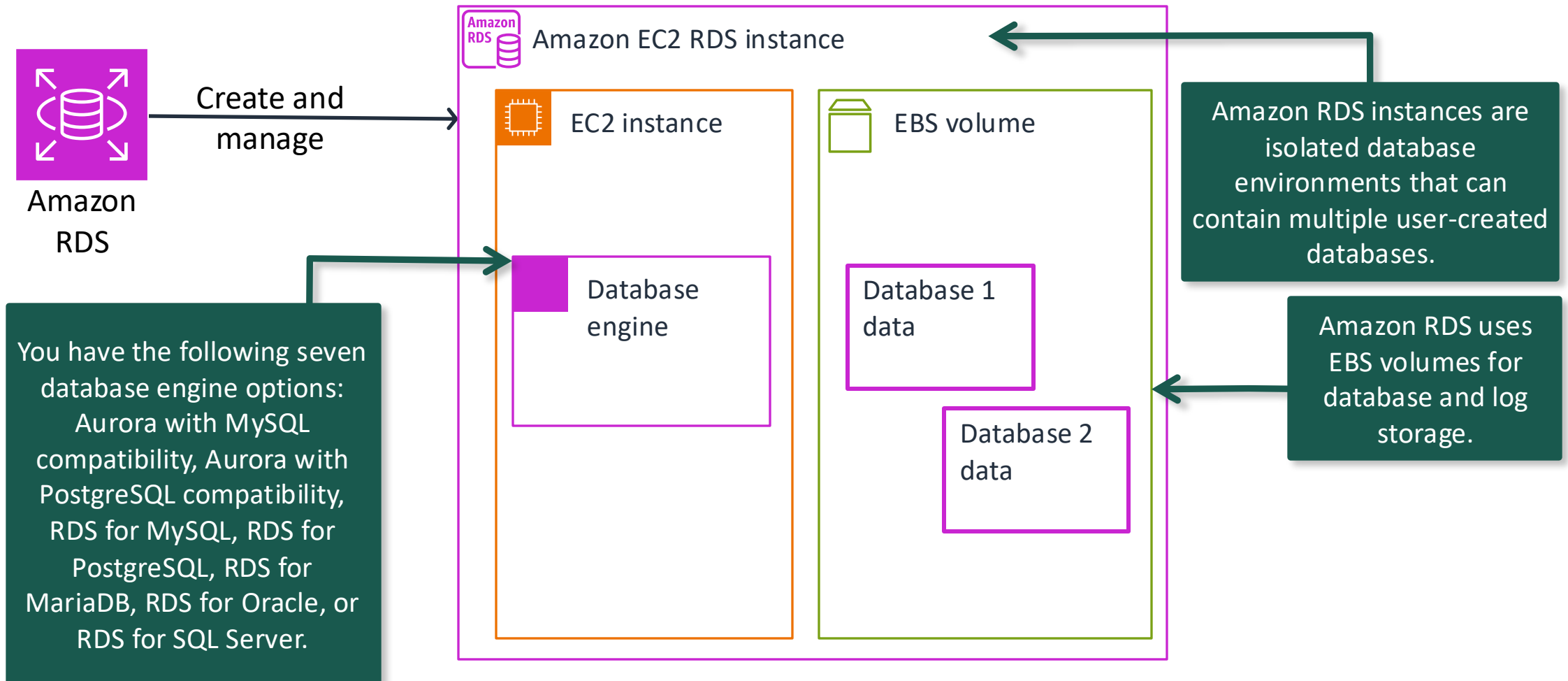
You can configure automated backups, database snapshots, and automatic host replacement.



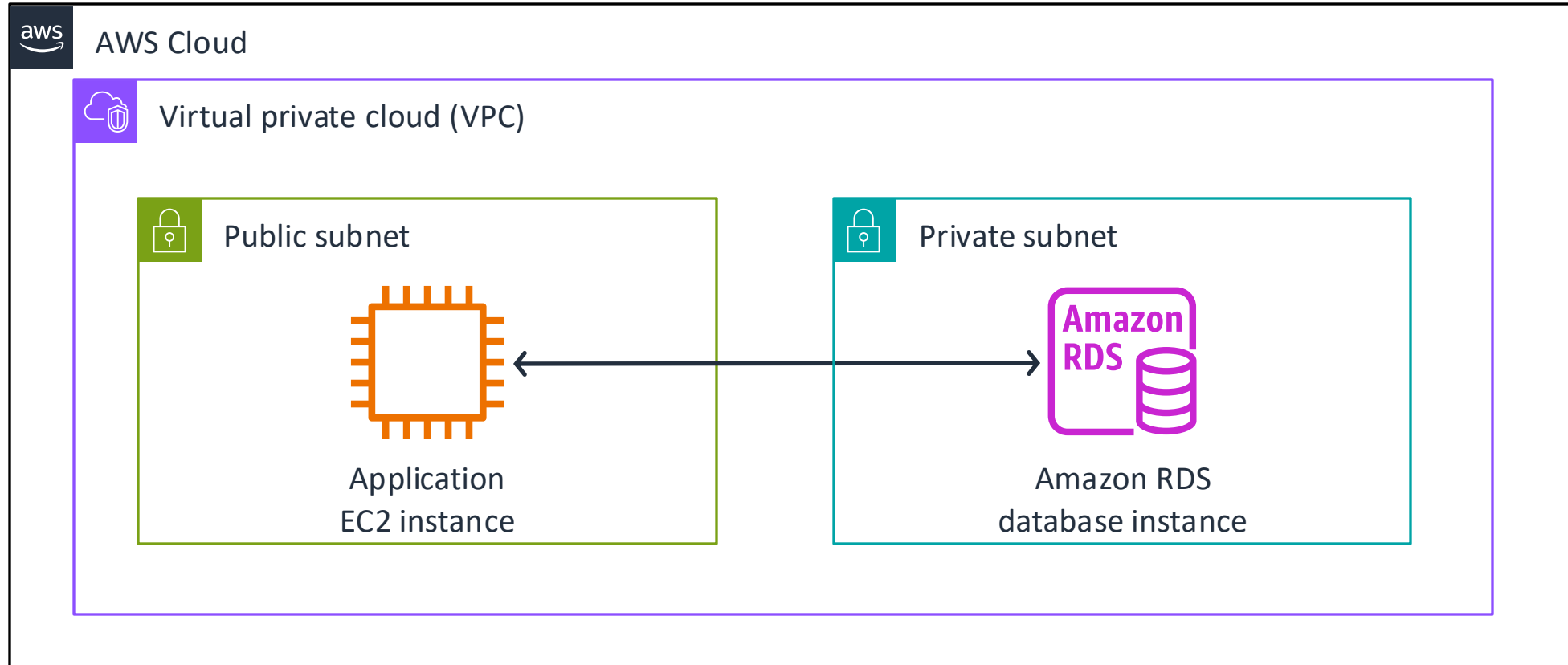
Secure and compliant

You can isolate your database in your own virtual network.

Amazon RDS database architecture



Architecture diagram of a database layer



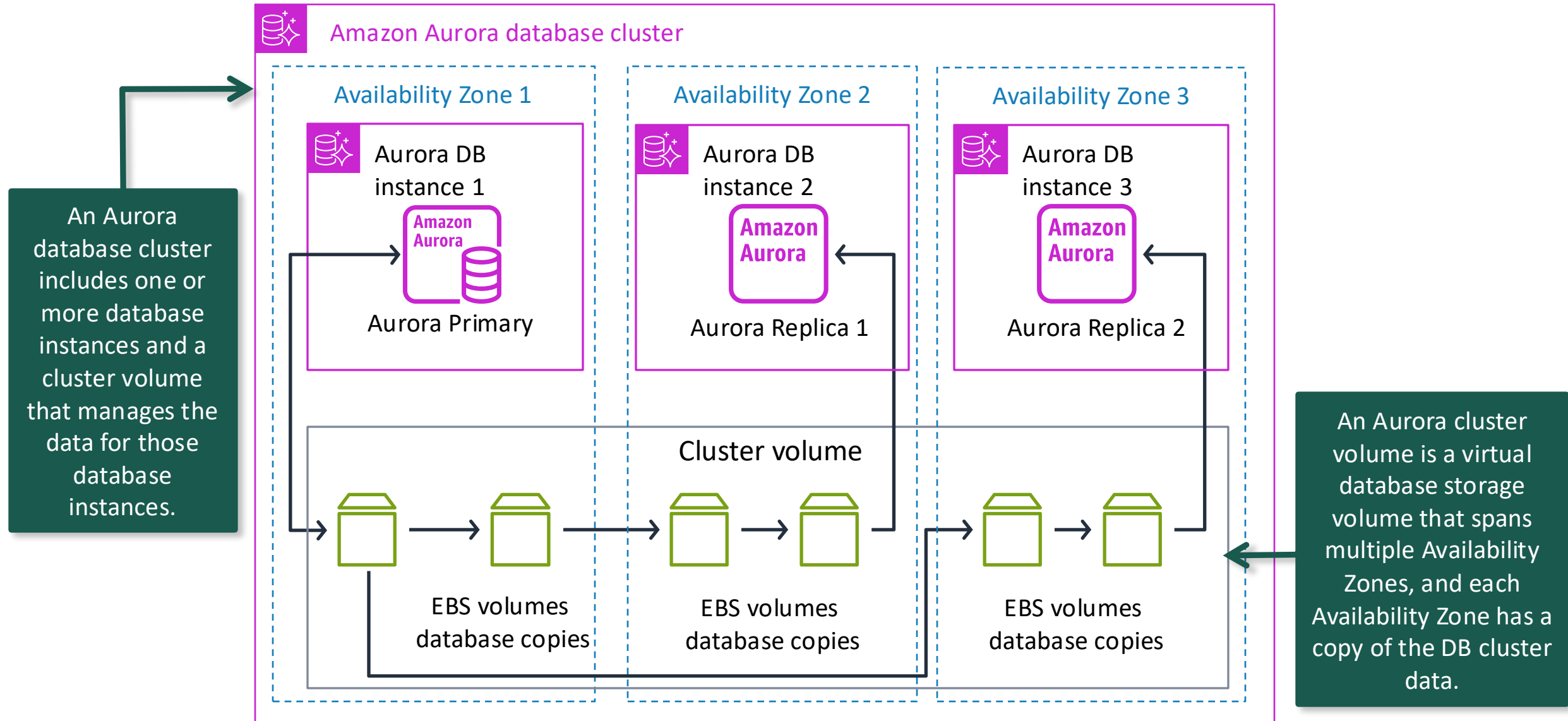
Aurora



Aurora

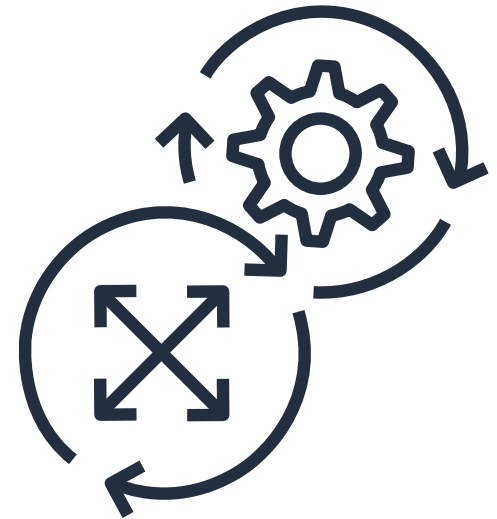
- Is a relational database management system (RDBMS) built for the cloud with full MySQL and PostgreSQL compatibility
- Is managed by Amazon RDS
- Provides high performance and availability at one-tenth of the cost
- Delivers Multi-AZ deployments with Aurora Replicas

Aurora database clusters

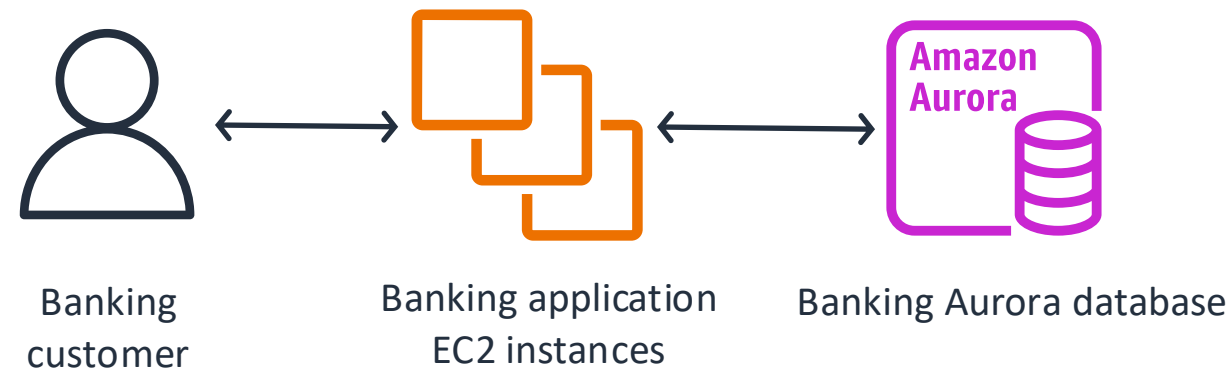


Aurora Serverless

- Is an on-demand auto scaling configuration for Aurora
- Provides hands-off capacity management
- Provides fine-grained scaling
- Is suitable for the following:
 - Variable workloads
 - New applications
 - Development and testing
 - Capacity planning



Amazon RDS use case: Banking transactions



Transaction ID	Date	Transaction Description	Transaction Type	Transaction Amount
0079834514	2023-11-05	Utility	Withdrawal	100.00
0079834513	2023-11-05	Employer name	Direct deposit	1000.00
0079834512	2023-11-04	Interest payment	Deposit	0.07

Amazon RDS EC2 instance types and sizing

- General purpose (T4g, T3, M6g, and M5)
- Memory-optimized (R6g, R5, X2g, and X1E)

Instance type	Memory (GiB)	vCPU
db.m6g.large	8	2
db.r6g.large	16	2
db.m6g.xlarge	16	4
db.r6g.xlarge	32	4

If m6g.large is the instance that needs an upgrade, first identify the issue.

Does it need more memory or CPU?

mg6.xlarge would provide the CPU upgrade.

rg6.large would provide the memory upgrade.

Amazon RDS security best practices



Run your DB instance in a VPC for the greatest possible network access control.



Use SSL or TLS connections with database instances running certain database engines.



Use AWS Identity and Access Management (IAM) policies to assign permissions for managing Amazon RDS resources.



Encrypt database instances and snapshots at rest with an AWS Key Management Service (AWS KMS) key.



Use security groups to control connecting IP addresses and resources.



Use the security features of your database engine to control database access.

Key takeaways: Amazon RDS



- Amazon RDS is a managed relational database service that can deploy familiar database engines.
- Aurora is a managed relational database engine built for the cloud. Aurora Serverless provides support for Aurora on-demand auto scaling.
- Amazon RDS provides a selection of instance types that are optimized to fit different relational database use cases.
- Differences in performance, scalability, failover, storage, high availability, backup, and database versions will determine which relational database is the optimum selection.



Guided lab: Creating an Amazon RDS Database (Amazon RDS lab)

Amazon RDS lab tasks



- In this lab, you will perform the following main tasks:
 - Create an Amazon RDS database.
 - Configure web application communication with a database instance.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform.

Debrief: Amazon RDS lab

- When you created your RDS DB instance, why did you configure it as single database instance without a Multi-AZ deployment option?
- What did you need to do so that your application could connect to your database?



Amazon RDS proxy connection management

Adding a Database Layer

Amazon RDS Proxy

Fully managed, highly available database proxy for Amazon RDS

More scalable

Pools and shares database connections for improved application scaling

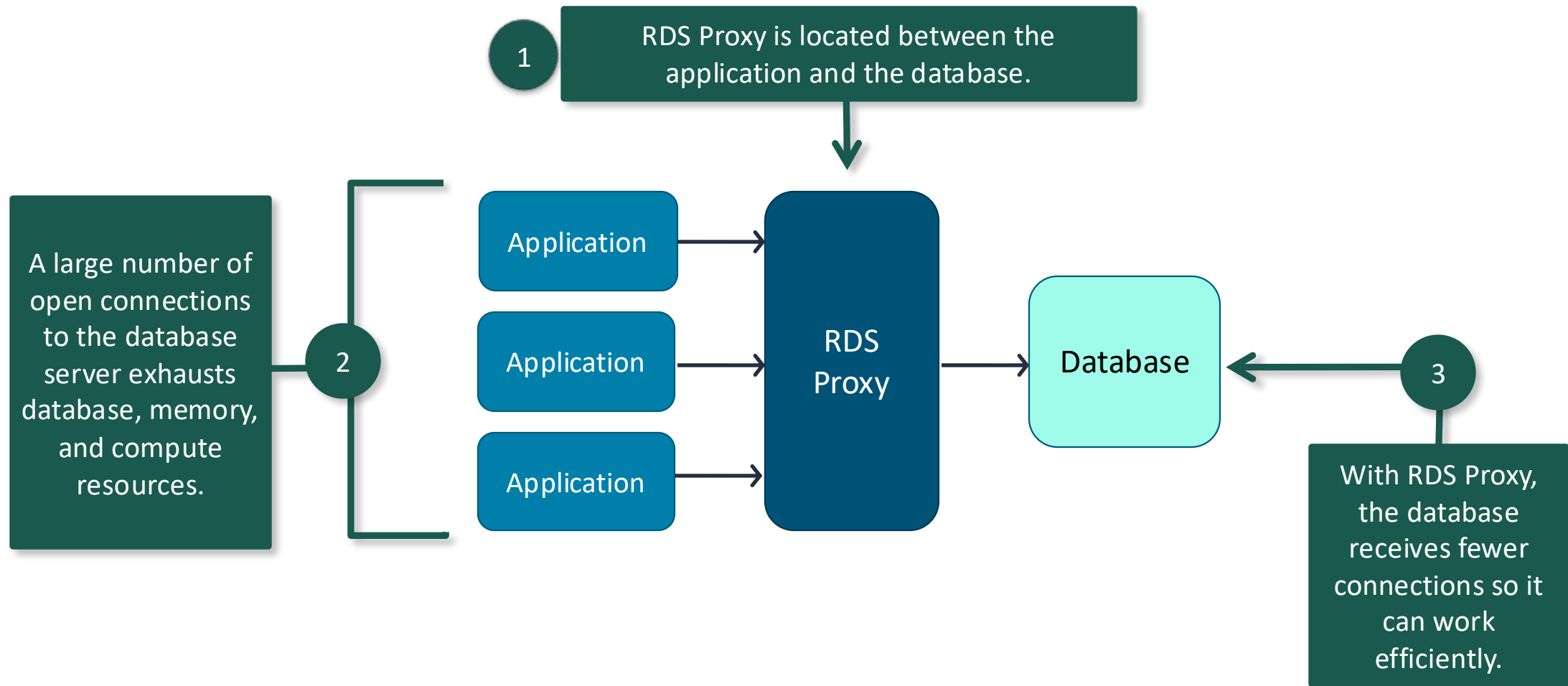
More resilient

Reduces database failover times for Aurora and Amazon RDS databases by up to 66 percent for Amazon RDS Multi-AZ databases

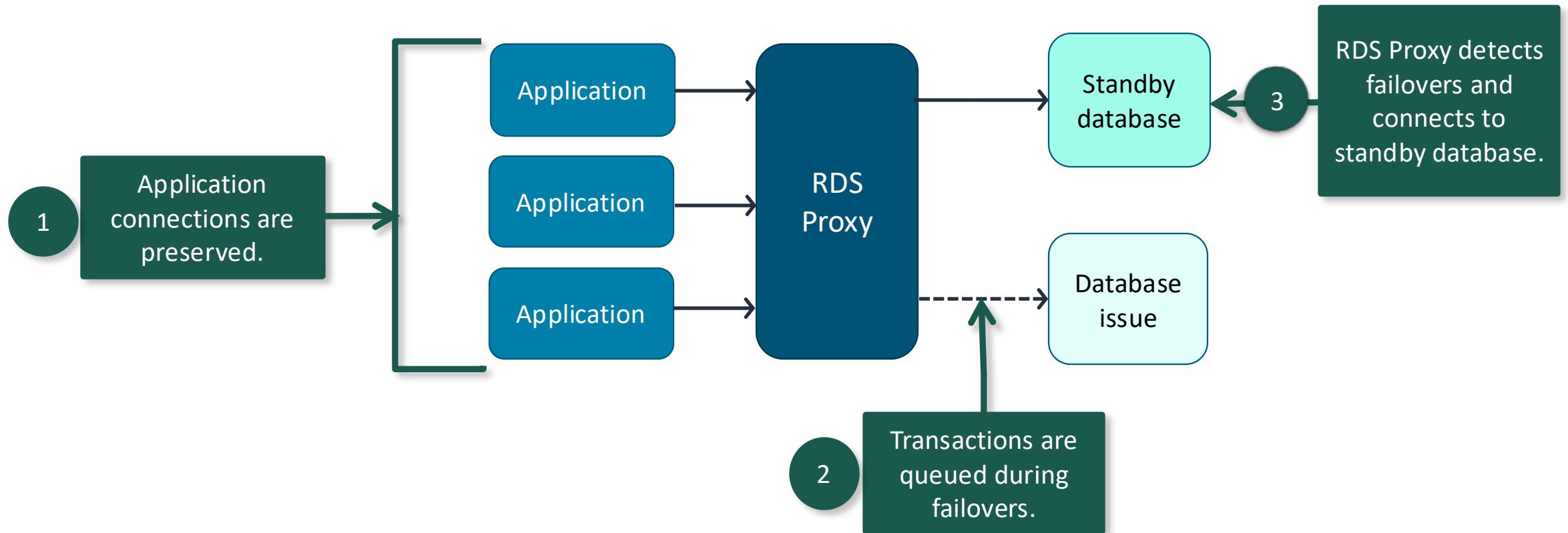
More secure

Enforces IAM authentication and stores credentials in AWS Secrets Manager

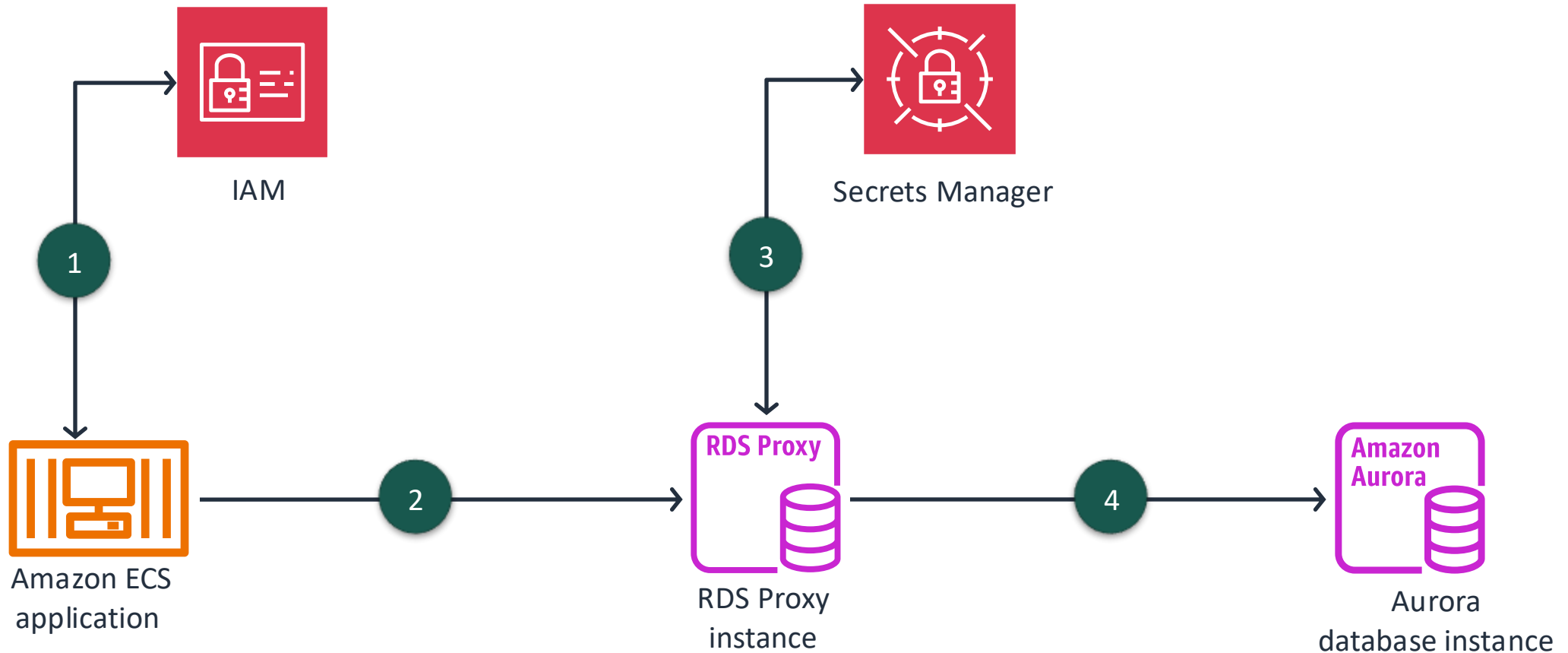
Connection pooling: Improved scalability



Seamless and fast failover: Improved availability



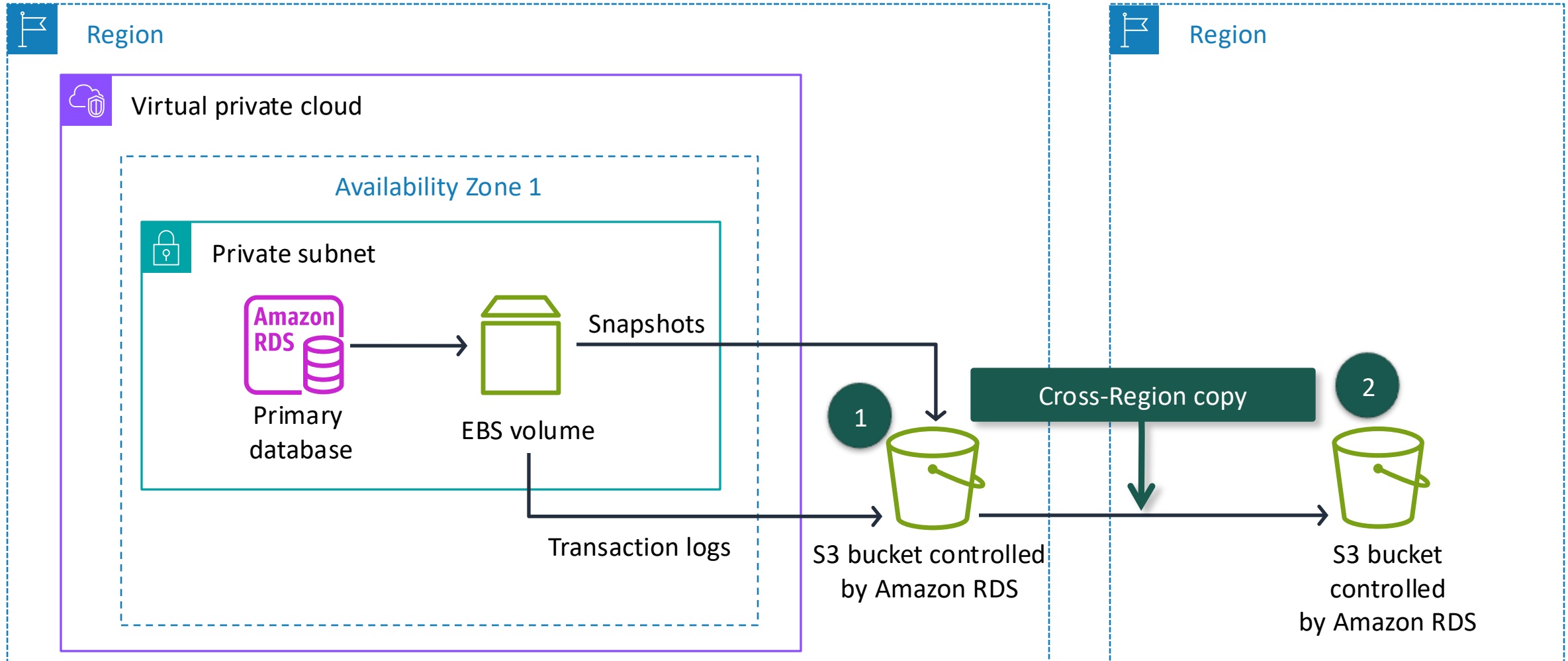
Streamlined authentication: Improved application security



Backing up data in Amazon RDS

Features	Automated Backups	Database Snapshots
Use Case	Restore a database instance to a specific point in time.	Back up a database instance in a known state, and then restore it to that specific state.
Backup Frequency	Daily during your backup window (transaction logs are captured every 5 minutes)	User-initiated (as frequently as the user chooses)
Retention Period	The default is 7 days but it can be set to up to 35 days. The backups can be automatically deleted after any retention period.	Kept until the user explicitly deletes it
Sharing with Other AWS Accounts	Cannot be shared (needs to be copied to a manual snapshot first)	Can be shared (shared snapshots can be copied by other AWS accounts)

Amazon RDS cross-region backups

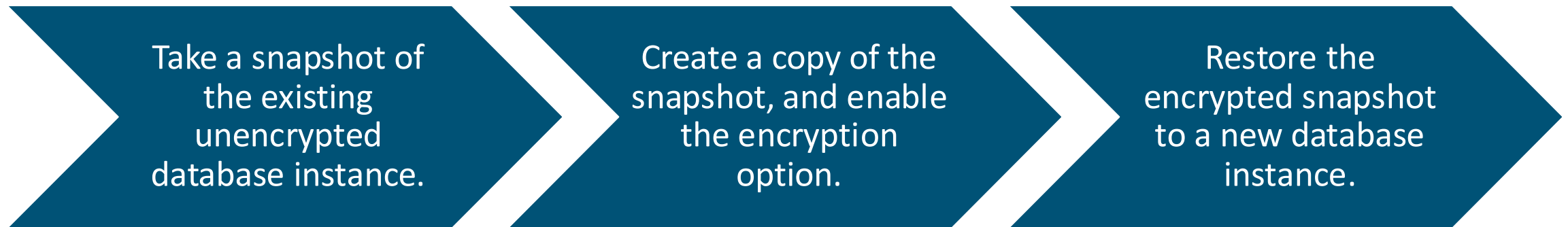


Amazon RDS encryption for backups

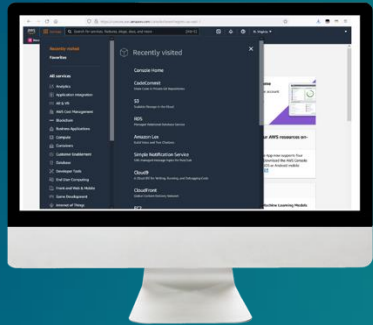
Amazon RDS can encrypt your RDS DB instances.

- Data at rest by using AWS KMS
- Data in transit by using SSL/TLS

Steps to back up an unencrypted database:



Demo: Amazon RDS Automated Backup and Read Replicas



- This demo uses Amazon RDS.
- In this demonstration, you will see how to:
 - Create a backup database schedule.
 - Create a database read replica.

Key takeaways: Amazon RDS proxy connection management



- RDS Proxy is a fully managed, highly available database proxy for Amazon RDS.
- By using a Multi-AZ approach, Amazon RDS synchronously replicates data to provide high availability. There is automatic failover to the standby instance.
- With read replicas, Amazon RDS asynchronously replicates data to provide high scalability. The standby instance can be manually promoted to stand alone.
- Amazon RDS provides different options for backing up and restoring database instances: automated backups and database snapshots. It offers encryption for data at rest and in transit.



Amazon DynamoDB

Adding a Database Layer

DynamoDB



DynamoDB

- Is a fully managed, serverless, NoSQL database
- Supports key-value and document data models
- Delivers millisecond performance and can automatically scale tables to adjust for capacity
- Is used for developing applications, mission-critical workloads that prioritize speed, scalability, and data durability

DynamoDB use cases



Develop software applications

Build internet-scale applications that support user-content metadata and caches that require high concurrency.



Create media metadata stores

Scale throughput and concurrency for media and entertainment workloads, such as real-time video streaming and interactive content.



Scale gaming platforms

Build out your game platform with player data, session history, and leaderboards for millions of concurrent users.

DynamoDB features

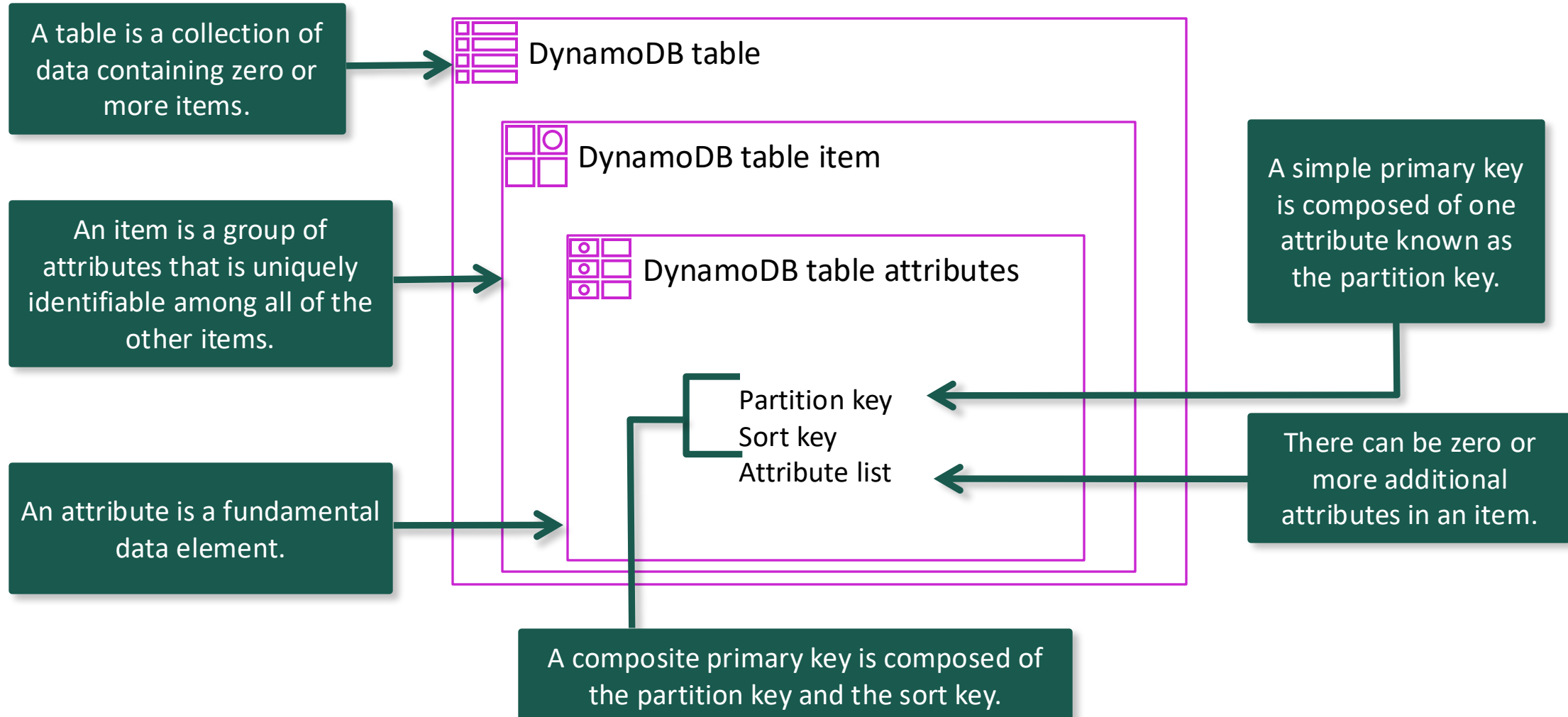
Serverless performance with limitless scalability

- Secondary indexes provide flexibility on how to access your data.
- Amazon DynamoDB Streams is ideal for an event-driven architecture.
- Multi-Region, multi-active data replication with global tables

Built-in security and reliability

- DynamoDB encrypts all customer data at rest by default.
- Point-in-time recovery protects data from accidental operations.
- DynamoDB allows fine-grained access control.

Amazon DynamoDB data structure



Amazon DynamoDB sample base table

Partition key = Device ID: 1	Sort key = Timestamp: 2023-11-20 15:42:00	Attribute = Temperature: 41.9	Attribute = Error Status: Low
Partition key = Device ID: 1	Sort key = Timestamp: 2023-11-20 15:42:30	Attribute = Temperature: 42	
Partition key = Device ID: 1	Sort key = Timestamp: 2023-11-20 15:43:00	Attribute = Temperature: 39	Attribute = Error Status: Low
Partition key = Device ID: 2	Sort key = Timestamp: 2023-11-20 15:42:00	Attribute = Temperature: 47	Attribute = Error Status: Low
Partition key = Device ID: 2	Sort key = Timestamp: 2023-11-20 15:42:30	Attribute = Temperature: 49	Attribute = Error Status: High
Partition key = Device ID: 2	Sort key = Timestamp: 2023-11-20 15:43:00	Attribute = Temperature: 46.9	

Alternate schema using a global secondary index

Base table

Partition key = Device ID: 1	Sort key = Timestamp: 2023-11-20 15:42:00	Attribute = Temperature: 41.9	Attribute = Error Status: Low
Partition key = Device ID: 1	Sort key = Timestamp: 2023-11-20 15:42:30	Attribute = Temperature: 42	
Partition key = Device ID: 2	Sort key = Timestamp: 2023-11-20 15:42:30	Attribute = Temperature: 49	Attribute = Error Status: High
Partition key = Device ID: 2	Sort key = Timestamp: 2023-11-20 15:43:00	Attribute = Temperature: 46.9	

Global Secondary Index (GSI)
[Eventual consistency]

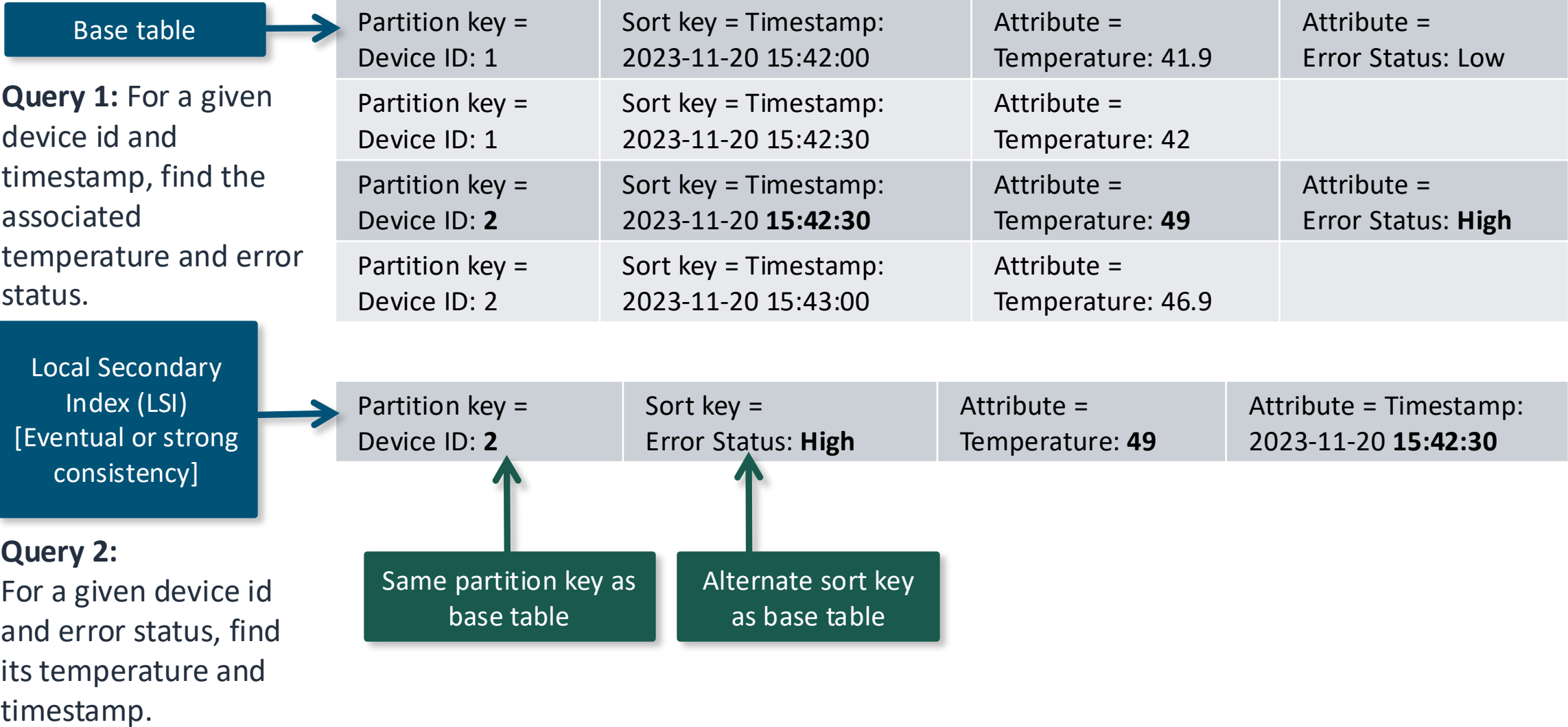
Partition key = Temperature: 49	Sort key = Device ID: 2	Attribute = Timestamp: 2023-11-20 15:42:30	Attribute = Error Status: High
--	--------------------------------	---	---------------------------------------

Alternate partition key as base table

Alternate sort key as base table

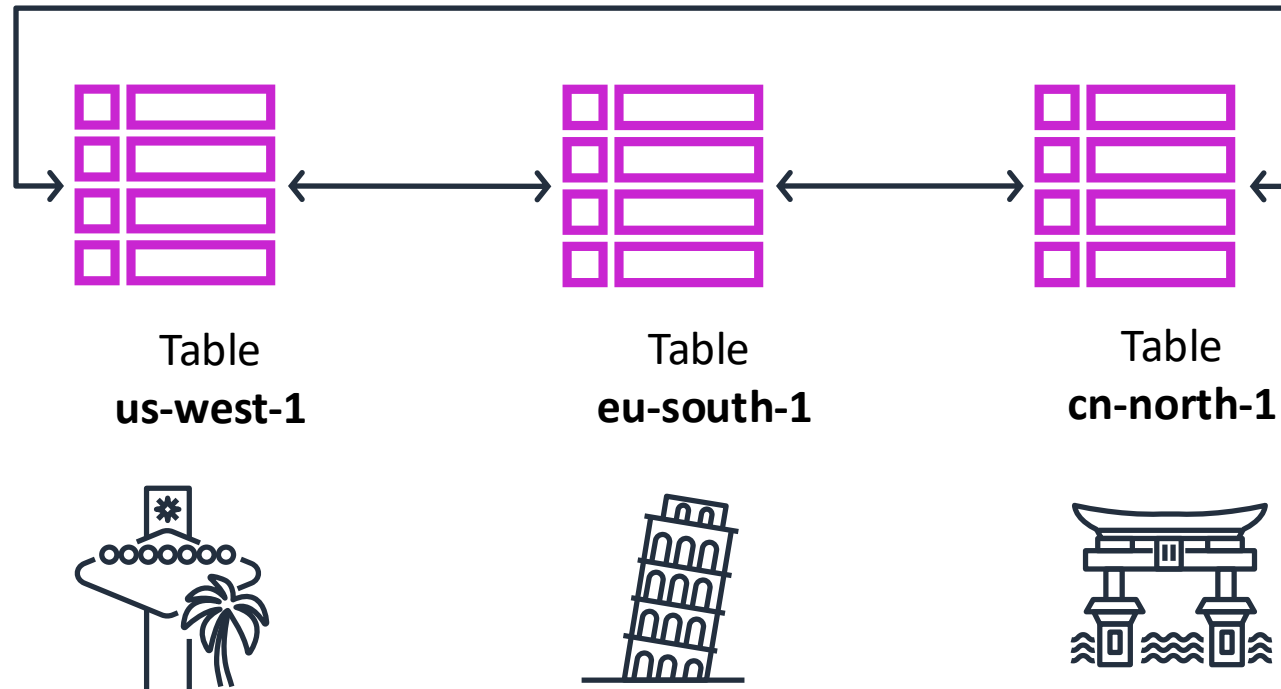
Query 2:
For a given temperature and device id, find the associated error status and timestamp.

Alternate schema using a local secondary index



Multi-region replication: Amazon DynamoDB global tables

Global tables provide a multi-region, multi-active database for fast local read and write performance for global applications.



DynamoDB security best practices

Preventative

- Use IAM roles to authenticate access.
- Use IAM policies for DynamoDB base authorization.
- Use IAM policy conditions for fine-grained access control.
- Use a VPC endpoint and policies to access DynamoDB.

Detective

- Use AWS CloudTrail to monitor AWS managed AWS KMS key usage.
- Monitor DynamoDB operations by using CloudTrail.
- Monitor DynamoDB configuration with AWS Config.
- Monitor DynamoDB compliance with AWS Config rules.

Key takeaways: Amazon DynamoDB



- DynamoDB is a fully managed, serverless, NoSQL database that supports key-value and document data models.
- DynamoDB offers global and local secondary indexes to provide flexibility on how to access your data.
- Global tables provide a multi-Region, multi-active database for fast performance for global applications.
- DynamoDB encrypts all user data at rest stored in tables, indexes, and streams. Preventative and detective security best practices help ensure data protection.



Purpose-built databases

Adding a Database Layer

The evolution of purpose-built databases

Opportunity	Database Evolution	AWS Examples
Improve on hierarchical databases' limited abilities to define relationships among data.	Relational	Amazon RDS
Improve performance by separating read-heavy reporting from an application's transactional database.	Data warehouse/ OLAP	Amazon Redshift
Analyze the more varied types of data being generated in large amounts on the internet.	Non-relational	DynamoDB
Take advantage of cloud computing's freedom to scale data stores up and down based on actual usage, and the ease of connecting different microservices to purpose-built data stores.	Purpose-built: • Document • Wide-column • In-memory • Graph • Timeseries • Ledger	Fully managed database services: • Amazon DocumentDB • Amazon Keyspaces • MemoryDB • Neptune • Timestream • Amazon QLDB

Amazon Redshift



Amazon Redshift

- Is a fully managed, cloud-based data warehousing service designed to handle petabyte-scale analytics workloads
- Achieves optimum query performance with columnar storage
- Has an Amazon Redshift Serverless option
- Is used for online analytics processing (OLAP)

AWS fully managed purpose-built database options



Amazon DocumentDB (with
MongoDB compatibility)
Document database



Amazon Neptune
Graph database



Amazon Keyspaces (for
Apache Cassandra)
Cassandra workloads



Amazon Timestream
Timeseries database

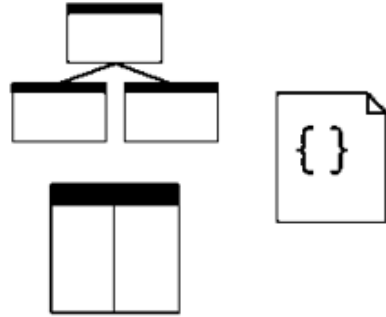


Amazon MemoryDB for
Redis
In-memory workloads



Amazon Quantum Ledger
Database (Amazon QLDB)
Ledger database

Matching a database to your business need



Suitable workloads

Analyze your workload requirements to see if they match the database's capabilities.

Data model

Understand the characteristics of the data model that you would need to use with the database.

Features and benefits

Familiarize yourself with key features and configuration options to optimize performance.

Common use cases

Review common use cases to find reference architectures and examples.

Amazon DocumentDB

Suitable workloads	Data model	Features and benefits	Common use cases
• Require a flexible schema for fast, iterative development • Need to store data that has different attributes and data values	• Uses a document data model • Stores and quickly accesses documents • Queries on any attribute	• Is a MongoDB-compatible, JSON document database • Is good for complex documents that are dynamic and require one-time querying, indexing, and aggregations	• Content management system (CMS) • Customer profiles • Storage and management of operational data from any source

Amazon Keyspaces

Suitable workloads	Data model	Features and benefits	Common use cases
• Fast querying capability of high volumes of data • Scalability and consistent performance on heavy write loads	• Is a wide column data model • Stores data in flexible columns, which permits data to evolve over time • Partitions across distributed database systems	Scalable, highly available, and managed Apache Cassandra-compatible database service	Process data at high speeds for applications that require millisecond latency: • Industrial equipment maintenance • Trade monitoring • Route optimization

MemoryDB

Suitable workloads	Data model	Features and benefits	Common use cases
• Latency-sensitive workloads • High request rate • High data throughput • No data loss	• Is an in-memory database service • Relies primarily on memory for data storage • Minimizes response time by eliminating the need to access disks	Stores an entire dataset in memory and leverages a distributed transactional log to provide the following: • In-memory speed and consistency • Data durability and recoverability	• Caching • Game leaderboards • Bank user transactions

Neptune

Suitable workloads	Data model	Features and benefits	Common use cases
• Find connections or paths in data • Combine data with complex relationships across data silos • Navigate highly connected datasets, and filter results based on certain variables	• Is a graph data model • Stores data and the relationship of that data to other data as equally important to the data itself • Quickly creates and navigates relationships between data	• Supports graph applications that require high throughput and low latency graph queries • Creates and navigates data relations quickly	• Recommendation engines • Fraud detection • Knowledge graphs • Drug discovery • Social networking

Timestream

Suitable workloads	Data model	Features and benefits	Common use cases
• Identify patterns and trends over time • Determine value or performance over time • Rely on efficient data processing and analytics • Require ease of data management	• Is a timeseries data model • Is a sequence of data points recorded over a time interval for measuring events that change over time • Provides the ability to collect, store, and process data sequenced by time	• Simplifies timeseries data access • Has built-in timeseries functions for quick analysis of timeseries data by using SQL	• Identify trends from data generated by Internet of Things (IoT) applications • Improve performance by analyzing web traffic data for your applications in real time

Amazon QLDB

Suitable workloads	Data model	Features and benefits	Common use cases
• Maintain an accurate history of application data • Track the history of financial transactions • Verify the data lineage of claims	• Is a ledger database • Provides an immutable and verifiable history of all changes to application data	• Provides a transparent, immutable, and cryptographically verifiable transaction log • Provides built-in data integrity • Provides a consistent event store	• Storing financial transactions • Maintaining claim history

Key takeaways: Purpose-built databases



- Amazon Redshift is a cloud-based data warehousing service.
- Amazon DocumentDB is a MongoDB-compatible, JSON document database.
- Amazon Keyspaces is a wide-column database service that is compatible with Apache Cassandra.
- MemoryDB is a durable in-memory database service.
- Neptune is a high-performance graph database engine.
- Timestream is a purpose-built database for timeseries data.
- Amazon QLDB is a ledger database that maintains a verifiable log of data changes.



Migrating data into AWS databases

Adding a Database Layer

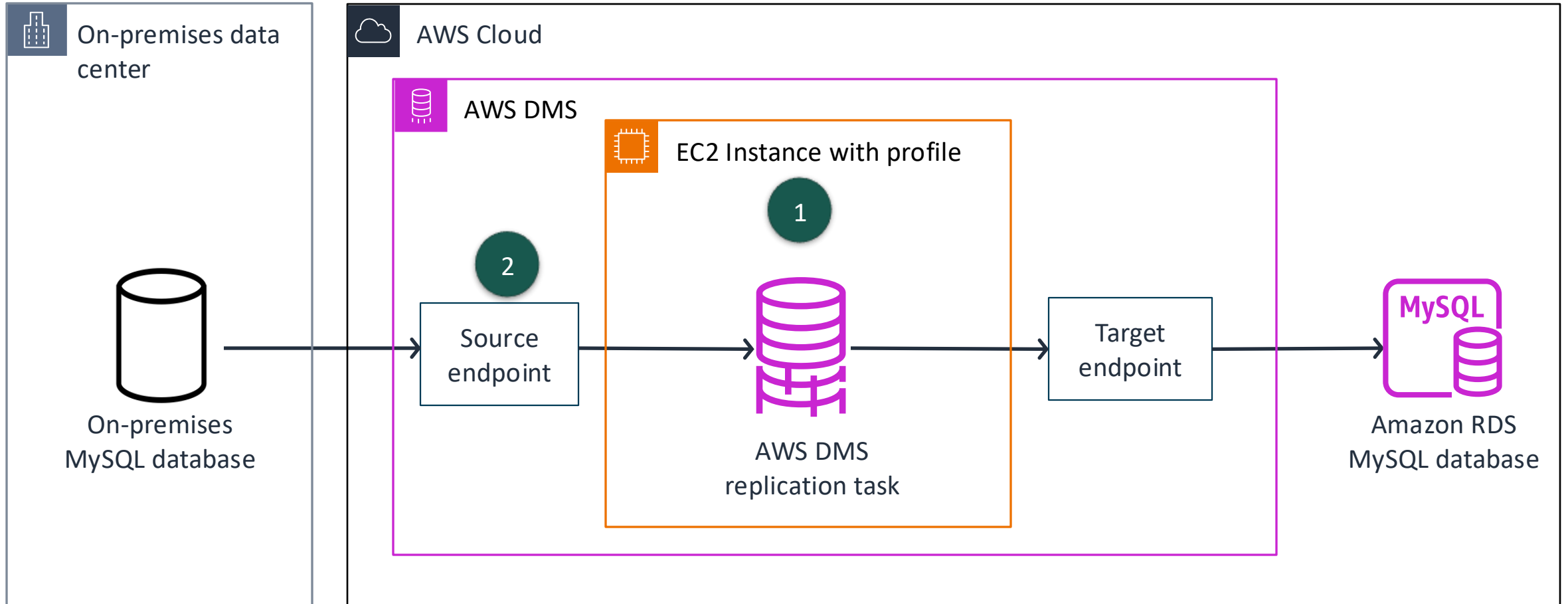
AWS DMS



AWS DMS

- Is a managed migration and replication service
- Helps move existing database and analytics workloads to and within AWS
- Supports most widely used commercial and open source databases
- Replicates data on demand or on a schedule to replicate changes from a source

AWS DMS homogeneous migration



Tools for heterogeneous database migrations

Database discovery tool

AWS DMS Fleet Advisor

Automatically inventories and assesses on-premises database and analytics server fleet

Schema conversion tools

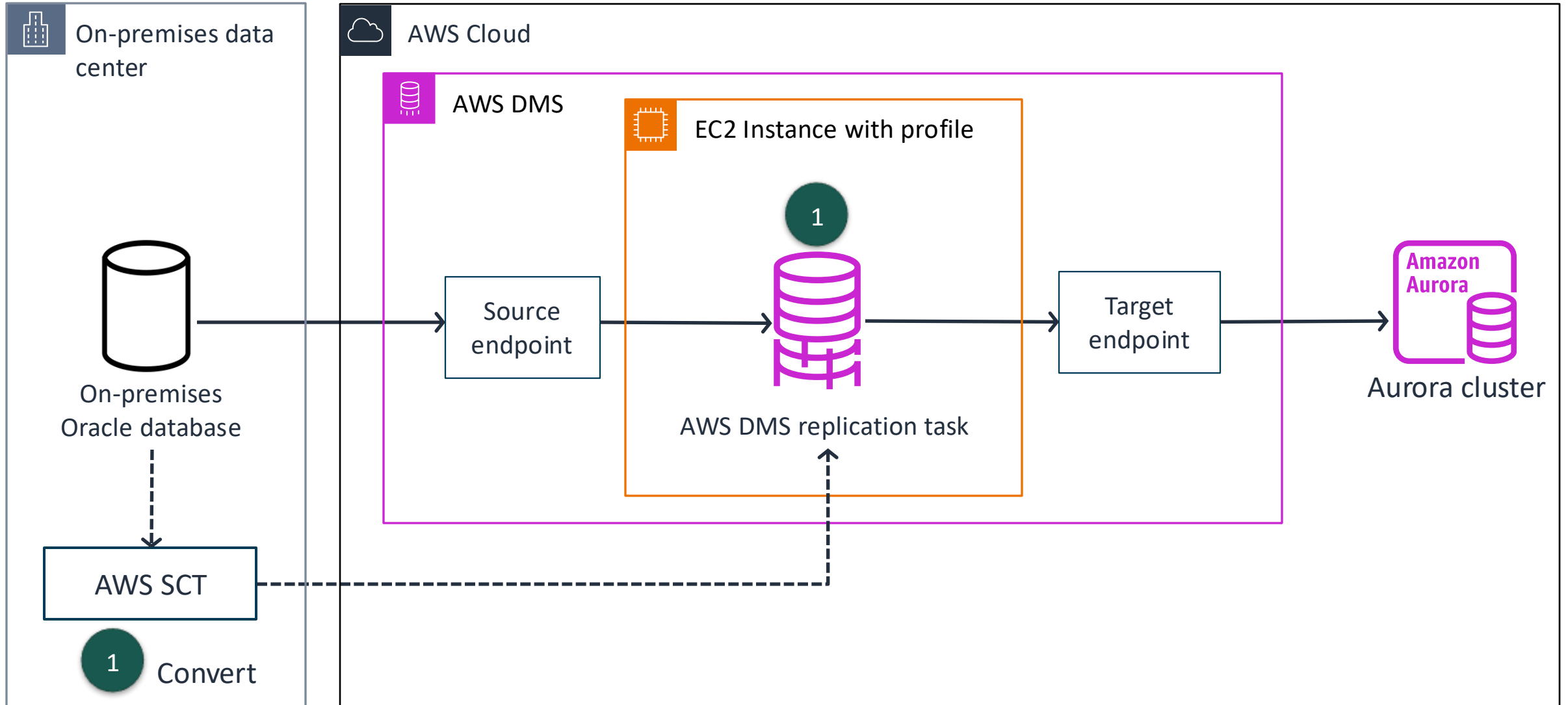
AWS Schema Conversion Tool (AWS SCT)

Converts your source schema and SQL code into an equivalent target schema and SQL code

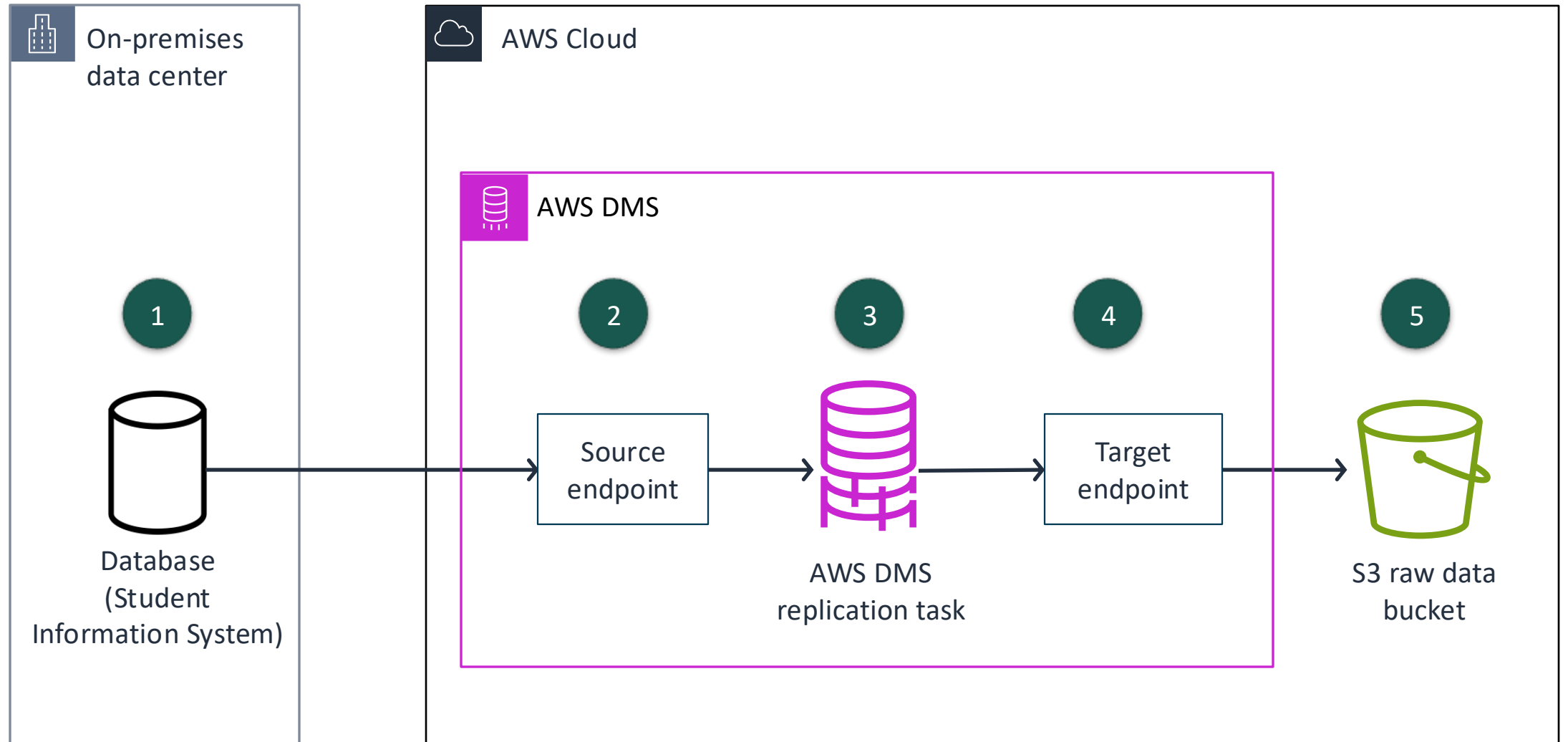
AWS DMS Schema Conversion

Is a centrally managed service for schema assessment and conversion available within AWS DMS workflows

AWS DMS heterogeneous migration with AWS SCT



AWS DMS replicates data from a database into a data lake



Key takeaways: Migrating data into AWS databases



- AWS DMS is a managed migration and replication service that helps you move your databases and analytics workloads to AWS quickly and securely.
- You can migrate between source and target endpoints that use the same database engine (homogenous migration) and migrate between source and target endpoints that use different database engines (heterogenous migration).
- AWS SCT and AWS DMS Schema Conversion are tools for schema and code conversion.



Café lab: Migrating a Database to Amazon RDS (Migration lab)

The evolving café architecture: Version 3

Architecture Version	Business Reason for Update	Technical Requirements and Architecture Update
V1	Create a static website for a small business.	Host the website on Amazon S3.
V2	Add online ordering.	Deploy the web application and database on Amazon EC2.
V3	Reduce the effort to maintain the database and secure its data.	Separate the web and database layers. Migrate the database to Amazon RDS on a private subnet.
V4	Enhance the security of the web application.	Use Amazon VPC features to configure and secure public and private subnets.
V5	Create separate access mechanisms based on role.	Add IAM groups and attach resource policies to application resources. Add IAM users to groups based on role.
V6	Ensure that the website can handle an expected increase in traffic.	Add a load balancer, implement auto scaling on the EC2 instances, and distribute compute and database instances across two Availability Zones.



Migration lab tasks



- In this lab, you will perform the following main tasks:
 - Create an RDS DB instance.
 - Import data from the EC2 DB instance to the RDS DB instance.
 - Connect the café application to the new database.
- Open your lab environment to start the lab and find additional details about the tasks that you will perform.

Debrief: Migration lab

- When you created your Amazon RDS MariaDB database, why did you not create a standby instance?
- Where did you get the password that you used to connect to the MariaDB database that runs on the EC2 instance?



Applying AWS Well-Architected Framework principles to the database layer

Adding a Database Layer

AWS Well-Architected Framework database pillars



Performance
Efficiency



Cost
Optimization



Security

Best practice approach: Architecture Selection



Performance
Efficiency

Best practices

Evaluate how trade-offs impact customers and architecture efficiency.

Use a data-driven approach for architectural choices.

Factor cost into architectural decisions.

Best practice approach: Data protection – protecting data at rest



Security

Best practice

Implement secure key management.

Enforce encryption at rest.

Best practice approach: Cost-effective resources – select the correct resource type, size, and number



Cost
Optimization

Best practice

Select the resource type, size, and number based on data.

Key takeaways: Applying AWS Well- Architected Framework principles to your database layer



- To achieve and maintain efficient workloads in the cloud, consider best practices in the performance efficiency pillar such as making selection choices based on data characteristics and access patterns.
- To secure your infrastructure effectively so that data is durable and safe from threats, consider best practices in the security pillar such as implementing secure key management and protecting data at rest.
- To meet the technical requirements of a workload with the lowest cost resource, consider best practices in the cost optimization pillar such as selecting the resource type, size, and number based on data.



Module wrap-up

Adding a Database Layer

Module summary

This module prepared you to do the following:

- Compare database types and services offered by Amazon Web Services (AWS).
- Explain when to use Amazon Relational Database Service (Amazon RDS).
- Describe the advanced features of Amazon RDS.
- Explain when to use Amazon DynamoDB.
- Identify AWS purpose-built database services.
- Describe how to migrate data into AWS databases.
- Design and deploy a database server.
- Use the AWS Well-Architected Framework principles when designing a database layer.

Module objectives

This module prepared you to do the following:

- Compare database types and services offered by Amazon Web Services (AWS).
- Explain when to use Amazon Relational Database Service (Amazon RDS).
- Describe the advanced features of Amazon RDS.
- Explain when to use Amazon DynamoDB.
- Identify AWS purpose-built database services.
- Describe how to migrate data into AWS databases.
- Design and deploy a database server. Use the Well-Architected Framework principles when designing a database layer.

Considerations for the café

- Discuss how the café lab in this module answered the key questions and decisions presented at the start of this module for the café business.



Module knowledge check



- The knowledge check is delivered online within your course.
- The knowledge check includes 10 questions based on material presented on the slides and in the slide notes.
- You can retake the knowledge check as many times as you like.

Sample exam question

An application requires a highly available relational database with an initial storage capacity of 8 TB. The database will grow by 8 GB every day. To support expected traffic, at least eight read replicas will be required to handle database reads. Which option will meet these requirements?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- Highly available relational database
- Grow by 8 GB every day
- Read replicas

Sample exam question: Response choices

An application requires a highly available relational database with an initial storage capacity of 8 TB. The database will grow by 8 GB every day. To support expected traffic, at least eight read replicas will be required to handle database reads. Which option will meet these requirements?

Choice	Response
A	Amazon DynamoDB
B	Amazon Neptune
C	Amazon Aurora
D	Amazon Redshift

Sample exam question: Answer

The answer is C.

Choice	Response
A	
B	
C	Amazon Aurora
D	



Thank you

Corrections, feedback, or other questions?

Contact us at <https://support.aws.amazon.com/#/contacts/aws-academy>.